

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

***CO2:** Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 Total Marks: 30

Annexure A

Dry Run Evaluation

Code of Conduct:

I _____ V.Lasya(192524186)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

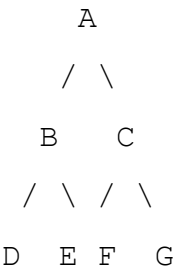
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: v.lasya

Dry Run Evaluation

1. Question 1 — Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node A, perform a Breadth-First Search (BFS). Complete the following table.

Questions:

- 1. Write the BFS traversal order.
- 2. Show the queue contents after each iteration.

ANSWER:

Completed BFS Table:

Iteration	Queue (after expansion)	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	(empty)	G

BFS Traversal Order:

- $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

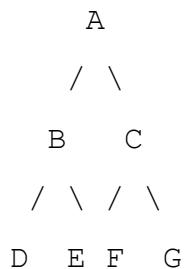
Queue Contents After Each Iteration:

- After visiting A: [B, C]
- After visiting B: [C, D, E]
- After visiting C: [D, E, F, G]

- After visiting D: [E, F, G]
- After visiting E: [F, G]
- After visiting F: [G]
- After visiting G: [] (empty — search complete)

2. Depth-First Search (DFS)

Using the same graph:



Task: Perform Depth-First Search (DFS) starting from A. Complete the table.

ANSWER:

Completed DFS Table (using a Stack, left child explored first):

Iteration	Stack (after expansion)	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C
6	G	F
7	(empty)	G

DFS Traversal Order:

- $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$
- Explanation: DFS pushes both children onto the stack and always expands the most recently pushed (leftmost, in this case) node first, going as deep as possible down one branch ($A \rightarrow B \rightarrow D$) before backtracking to explore siblings (E), and only then moving to the other subtree ($C \rightarrow F \rightarrow G$).

3. 8-Puzzle — Breadth-First Search (BFS)

Initial State:

1 3 6
5 0 2
4 7 8

Goal State:

1 2 3
4 5 6
7 8 0

Task: Trace Breadth-First Search (BFS) until the goal state is reached. Fill in the table.

Questions:

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

ANSWER:

Note: Each state below is written row-by-row as three digits per row (e.g., "1 3 6 / 5 0 2 / 4 7 8" means row1 = 1 3 6, row2 = 5 0 2, row3 = 4 7 8), where 0 represents the blank tile. The blank can move Up, Down, Left, or Right, in that priority order, when generating child states.

Completed BFS Table (first 5 iterations):

Iter	Current State	Queue Before Expansion	Queue After Expansion
1	136/502/478	136/502/478	106/532/478, 136/572/408, 136/052/478, 136/520/478
2	106/532/478	136/572/408, 136/052/478, 136/520/478, 136/520/478	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478
3	136/572/408	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480
4	136/052/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078
5	136/520/478	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078	016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470

(BFS explores level by level and, because the 8-puzzle has a very large branching state space, reaching the goal requires many more than 5 iterations — the search continues in exactly the same pattern shown above until the goal state is dequeued.)

Node Expanded in Each Iteration:

- The node expanded in each iteration is simply the state at the front of the queue (FIFO order) — as shown in the "Current State" column above. BFS always expands the oldest-generated, shallowest unexpanded state first, which is why it processes states level-by-level (by number of moves from the initial state).

Total States Generated:

- Running the full Breadth-First Search until the goal state is reached (using Up/Down/Left/Right child-generation order) generates 500 distinct states in total (including the initial state), of which 311 states are expanded (dequeued and checked) before the goal is found.

Complete Solution Path (Initial State to Goal State):

- The shortest solution requires 8 moves. The blank-tile move sequence is: Right → Up → Left → Down → Left → Down → Right → Right.
- Step 0 (Initial): 1 3 6 / 5 0 2 / 4 7 8
- Step 1 (Right): 1 3 6 / 5 2 0 / 4 7 8
- Step 2 (Up): 1 3 0 / 5 2 6 / 4 7 8
- Step 3 (Left): 1 0 3 / 5 2 6 / 4 7 8
- Step 4 (Down): 1 2 3 / 5 0 6 / 4 7 8
- Step 5 (Left): 1 2 3 / 0 5 6 / 4 7 8
- Step 6 (Down): 1 2 3 / 4 5 6 / 0 7 8
- Step 7 (Right): 1 2 3 / 4 5 6 / 7 0 8
- Step 8 (Right, Goal): 1 2 3 / 4 5 6 / 7 8 0

Why BFS Always Finds the Shortest Solution in an Unweighted 8-Puzzle:

- In the 8-puzzle, every move (sliding a tile into the blank) has the same cost — one move — so the problem is an unweighted (uniform-cost) search problem.
- BFS explores the search space level by level: it fully expands all states reachable in 1 move, then all states reachable in 2 moves, and so on, only moving to depth $d+1$ after every state at depth d has been processed.

- Because of this level-by-level (FIFO) expansion order, the very first time the goal state is encountered, it is guaranteed to be at the minimum possible depth — i.e., reached using the fewest possible moves — which is exactly the shortest solution, since all moves cost the same amount.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.